H5: Loops deel 2

Labo 2 · week 6 · debuggen

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Wat is debuggen?** En wat niet?
2. **De debugger van Visual Studio**: breakpoints, stappen en variabelen bekijken
3. **Demo**: een gemiddelde dat niet klopt
4. **Zoek de bug**: drie programma's met een fout
5. **Aan de slag**: H5 afwerken, mét debugger

🐞 *De eerste "bug" was echt een mot, in 1947 vastgelopen in een computer. Sindsdien zoeken we.*

Wat is debuggen?

Je programma draait, maar het klopt niet

Drie soorten fouten:

- **Compileerfout**: rode kronkellijn, het programma start niet. Visual Studio zegt waar het fout zit.
- **Crash**: het programma stopt met een foutmelding, bijvoorbeeld bij `Convert.ToInt32("abc")`.
- **Logische fout**: het programma draait, maar het **resultaat is fout**. Niemand waarschuwt je.

De **debugger** helpt vooral bij die laatste soort.



Een logische fout is als een gps die zelfverzekerd zegt "u bent gearriveerd" in het midden van een weiland.

Wat doet een debugger?

De naam is misleidend: een debugger **verwijdert geen bugs**.

Hij laat je je programma **stap voor stap** uitvoeren, en op elk moment **kijken** welke waarden je variabelen hebben.

> **BELANGRIJK**
>
> Debuggen is **informatie verzamelen**. Je ziet wat je programma **echt** doet, in plaats van wat je **denkt** dat het doet.

Debuggen: twee keer zo moeilijk als code schrijven. Schrijf dus geen code die slimmer is dan jezelf.

De debugger

1. Een breakpoint zetten

Een **breakpoint** is een plaats waar je programma moet **pauzeren**.

- Klik in de **grijze marge** links van een regel, of zet je cursor op de regel en druk **F9**.
- Er verschijnt een **rood bolletje**.
- Nog eens klikken of F9 haalt het weg.



Een breakpoint: de pauzeknop die je lector nooit krijgt.

2. Starten en pauzeren

- Start met **F5** (of **Debug** → **Start Debugging**), niet met Ctrl + F5.
- Het programma loopt tot aan het breakpoint en **pauzeert** daar.
- Een **gele pijl** wijst naar de regel die **als volgende** uitgevoerd wordt. Die regel is dus **nog niet** uitgevoerd.



De gele pijl wijst naar wat nog moet komen. Net als je deadline.

3. Stappen

Toets	Naam	Wat doet het?
F10	Step Over	de volgende regel, als één geheel
F11	Step Into	**in** de methode op deze regel stappen
Shift + F11	Step Out	de methode afwerken en terugkeren
F5	Continue	verder tot het volgende breakpoint
Shift + F5	Stop	stoppen met debuggen

😈 *F10 stapt over, F11 stapt erin. F12 is voor wie verdwaald is.*

4. Variabelen bekijken

- **Hover** met je muis over een variabele: je ziet meteen de waarde.
- Het venster **Locals** toont alle variabelen die nu in scope zijn. Rood betekent: net veranderd.
- In het venster **Watch** typ je zelf een variabele of expressie, zoals `sum / count`.
- Vind je een venster niet? **Debug** → **Windows**.



Het Locals-venster: al je variabelen op een rij. Zoals een klasfoto, maar dan eerlijk.

5. Wie riep wie? De Call Stack

Je oefeningen zijn methoden die opgeroepen worden vanuit een menu. De **Call Stack** toont de weg ernaartoe:

Call Stack
`Loops.Average()` ← hier staat de gele pijl nu
`Loops.SubMenu()`
`Program.Main(string[] args)`

Bovenaan staat waar je nu bent. **Onderaan** staat wie het langst aan het uitvoeren is: `Main`.

De Call Stack: een roddel volgen tot aan de bron.

Demo

Een gemiddelde dat niet klopt

```
 1  int sum = 0;
 2  int count = 0;
 3  Console.WriteLine("Geef een getal (0 om te stoppen):");
 4  int number = Convert.ToInt32(Console.ReadLine());
 5  while (number != 0)
 6  {
 7      sum += number;
 8      count++;
 9      number = Convert.ToInt32(Console.ReadLine());
10  }
11  double average = sum / count;
12  Console.WriteLine($"Gemiddelde: {average}");
```

Invoer: `1`, `2` en `0`. We verwachten `1,5`.

```
Gemiddelde: 1
```

😈 *Het gemiddelde van 1 en 2 is 1? Volgens C# wel, als je hem zijn zin geeft.*

Op zoek met de debugger

1. Zet een **breakpoint** op `double average = sum / count;` (F9).
2. Start met **F5** en geef `1`, `2` en `0` in.
3. Kijk in **Locals**: `sum` is `3`, `count` is `2`. De loop werkt dus perfect.
4. Druk **F10**. Nu is `average` gelijk aan `1`. De fout zit in **deze ene regel**.
5. Typ in **Watch**: `sum / count`. Ook `1`. Want een `int` gedeeld door een `int` geeft een `int` (zie H2).

Oplossing: declareer `sum` als een `double`: `double sum = 0;`. Dan krijg je `1,5`.

😈 *Niet gokken, maar kijken. Zoals bij de frituur: eerst het bord lezen, dan bestellen.*

Zoek de bug

Bug 1: aftellen

```
1  Console.WriteLine("Vanaf welk getal aftellen?");
2  int start = Convert.ToInt32(Console.ReadLine());
3  for (int index = start; index > 1; index--)
4  {
5      Console.WriteLine(index);
6  }
7  Console.WriteLine("Start!");
```

Invoer `5`. Verwacht: `5 4 3 2 1 Start!`, maar we krijgen `5 4 3 2 Start!`.

Debugger: breakpoint in de loop, F10, en kijk naar `index` bij de laatste ronde. Na `2` wordt het `1`, en dan is `1 > 1` `false`.

Oplossing: `index >= 1`

😈 *Aftellen tot 2 en dan "Start!". De raket vertrekt een seconde te vroeg.*

Bug 2: de som van de even getallen

```
1  int sum = 0;
2  for (int index = 1; index <= 10; index++)
3  {
4      if (index % 2 == 1)
5      {
6          sum += index;
7      }
8  }
9  Console.WriteLine(sum);
```

Verwacht: `30` (2 + 4 + 6 + 8 + 10), maar we krijgen `25`.

Debugger: breakpoint op `sum += index;`. Je pauzeert bij `index` gelijk aan 1, 3, 5, … De oneven getallen dus.

Oplossing: `index % 2 == 0`

25 in plaats van 30. Bijna goed is ook fout. Vraag maar aan je bank.

Bug 3: het wachtwoord

```
1  Console.WriteLine("Wachtwoord?");
2  string password = Console.ReadLine();
3  int attempts = 1;
4  while (password != "gEhEiM")
5  {
6      Console.WriteLine("Fout! Probeer opnieuw.");
7      attempts++;
8  }
9  Console.WriteLine($"Gelukt na {attempts} pogingen.");
```

Na een fout wachtwoord staat het scherm vol met “Fout! Probeer opnieuw.” en kan je niets meer typen.

Debugger: breakpoint in de loop, een paar keer F5. `attempts` stijgt, maar `password` verandert **nooit**.

Oplossing: lees het wachtwoord opnieuw in de loop: `password = Console.ReadLine();`

😈 *“Fout! Probeer opnieuw.” Tot in de eeuwigheid. Klinkt als mijn eerste rijexamen.*

Handige extra's

- **Voorwaardelijk breakpoint**: rechtsklik op het rode bolletje → **Conditions**, bijvoorbeeld `index == 7`. Het programma pauzeert dan pas in ronde 7.
- **Run to Cursor** (**Ctrl + F10**): loop verder tot aan de regel waar je cursor staat, zonder extra breakpoint.
- **Waarden aanpassen**: in Locals kan je een waarde overschrijven, om te testen wat er dan gebeurt.



Een voorwaardelijk breakpoint: meteen doorspoelen naar de spannende scène.

Zo pak je het aan

1. **Voorspel**: wat zou er moeten gebeuren? Welke waarden verwacht je?
2. **Pauzeer**: zet een breakpoint vlak voor de plek waar het misgaat.
3. **Stap en kijk**: F10, en vergelijk elke waarde met je voorspelling.
4. **Gevonden**: waar verschilt wat je ziet van wat je verwachtte? Daar zit je bug.



Debuggen zonder te voorspellen is als een quiz kijken zonder mee te raden.

Samengevat

Toets	Wat?
F9	breakpoint aan of uit
F5	start debuggen, of verder tot het volgende breakpoint
F10	Step Over: volgende regel
F11	Step Into: in de methode stappen
Shift + F11	Step Out: de methode afwerken
Shift + F5	stoppen

Variabelen: **hover**, **Locals** en **Watch**. Wie riep wie: **Call Stack**.

🐞 *Een bug vinden is 90% van het werk. De andere 90% is hem oplossen.*

Aan de slag 💻

Werk de oefeningen van **H5** af, en gebruik de debugger als iets niet klopt:

1. **SumOfEvenNumbers**, **Factors** en **Accountant** (while)
2. **Fibonacci**, **PrimeNumberChecker** en **PrimeNumberGenerator** (for)
3. Alles van H5 wat nog niet af is

👉 Vastgelopen? **Eerst** een breakpoint zetten en kijken, **dan** pas je hand opsteken.

👹 `while (bugsLeft > 0) { Debug(); }`